KR MANGALAM UNIVERSITY

FLIGHT RESERVATION SYSTEM

DBMS CAPSTONE PROJECT REPORT

Submitted By: Kirti

Roll Number: 2501420011

Course / Subject: B.Tech CSE (Data Science) · DBMS

Faculty Name: Mrs. Swati Gupta

Submission Date: 10 November 2026

Submitted in partial fulfilment of the DBMS capstone project requirements.

Table of Contents

1. Introduction

2. Objectives

3. Software and Database

4. Database Schema

5. Main SQL Queries

6. Composite Indexes and EXPLAIN

7. Views

8. Window Functions

9. Screenshots and Execution Evidence

10. Conclusion

1. Introduction

The Flight Reservation System is a database project designed to manage flight details, passenger information, and bookings. It demonstrates relational tables, joins, views, composite indexes, query-plan inspection with EXPLAIN, and window functions for ranking and daily analysis.

2. Objectives

• Store flight routes, schedules, and seat capacity.

• Store passenger names and loyalty tiers.

• Maintain booking status, seat number, fare class, fare amount, and booking timestamp.

• Report seat availability, booking status, cancelled bookings, and possible overbooking.

• Improve common searches using composite indexes and inspect query plans with EXPLAIN.

• Create views for flight occupancy and waitlisted/confirmed bookings.

• Demonstrate ROW_NUMBER(), RANK(), running revenue, and LAG().

3. Software and Database

Database software: MySQL / MySQL Workbench.

Database name: FlightReservationDB.

The attached screenshots are included as evidence of the user's Workbench execution. The exact result values may depend on the current contents of the database.

4. Database Schema

Flights: flight_id (primary key), flight_number, source, destination, departure_date, departure_time, total_seats.

Passengers: passenger_id (primary key), passenger_name, email, loyalty_tier.

Bookings: booking_id (primary key), flight_id (foreign key), passenger_id (foreign key), seat_no, booking_status, fare_class, fare_amount, booking_timestamp.

Relationships: one flight can have many bookings, and one passenger can have many bookings. Bookings references Flights and Passengers through foreign keys.

5. Main SQL Queries

Seat availability: LEFT JOIN Flights and Bookings, count only Confirmed/Upgraded bookings, and subtract the count from total_seats. Cancelled and Waitlisted records are not counted as occupied seats.

Booking status: join Bookings with Passengers and Flights to display passenger, flight, seat, status, fare class, and fare amount.

Cancellation: UPDATE Bookings SET booking_status = 'Cancelled' for a selected booking. The project also includes a cancelled-bookings report.

Overbooking detection: compare the count of Confirmed/Upgraded bookings with total_seats; report the excess and a capacity status.

Note: the capacity-reduction step in the screenshots was a demonstration. The AI101 capacity was subsequently restored to 100.

6. Composite Indexes and EXPLAIN

Flight search index: CREATE INDEX idx_flight_search ON Flights (source, destination, departure_date). This matches the demonstrated search predicates for route and date.

Booking lookup index: CREATE INDEX idx_booking_lookup ON Bookings (flight_id, seat_no, booking_status). This supports lookups filtered by flight, seat, and status.

EXPLAIN was run before and after creating the flight search index and for a booking lookup. The optimizer may choose a table scan for very small tables; the key and rows columns should be interpreted from the actual Workbench output. A predicate such as YEAR(departure_date) = 2026 can make use of a date index less effective because the column is wrapped in a function.

7. Views

FlightOccupancy: reports each flight's total seats, confirmed/upgraded bookings, available seats, and occupancy percentage. The calculation protects against division by zero with NULLIF.

WaitlistedPassengers: joins booking, passenger, and flight details for records whose status is Waitlisted. Results are ordered by loyalty tier, fare class, and booking timestamp.

ConfirmedBookings: a simple view filtered to Confirmed bookings and defined with WITH CHECK OPTION. This option prevents changes made through the view from creating a row that no longer satisfies the view condition. Aggregate/join-based views are generally not directly updatable.

8. Window Functions

ROW_NUMBER(): assigns a unique priority number within each flight using PARTITION BY flight_number and ordering by loyalty tier, fare class, and booking timestamp.

RANK(): assigns the same rank to rows tied on the specified loyalty-tier and fare-class priority, and skips the next rank after a tie.

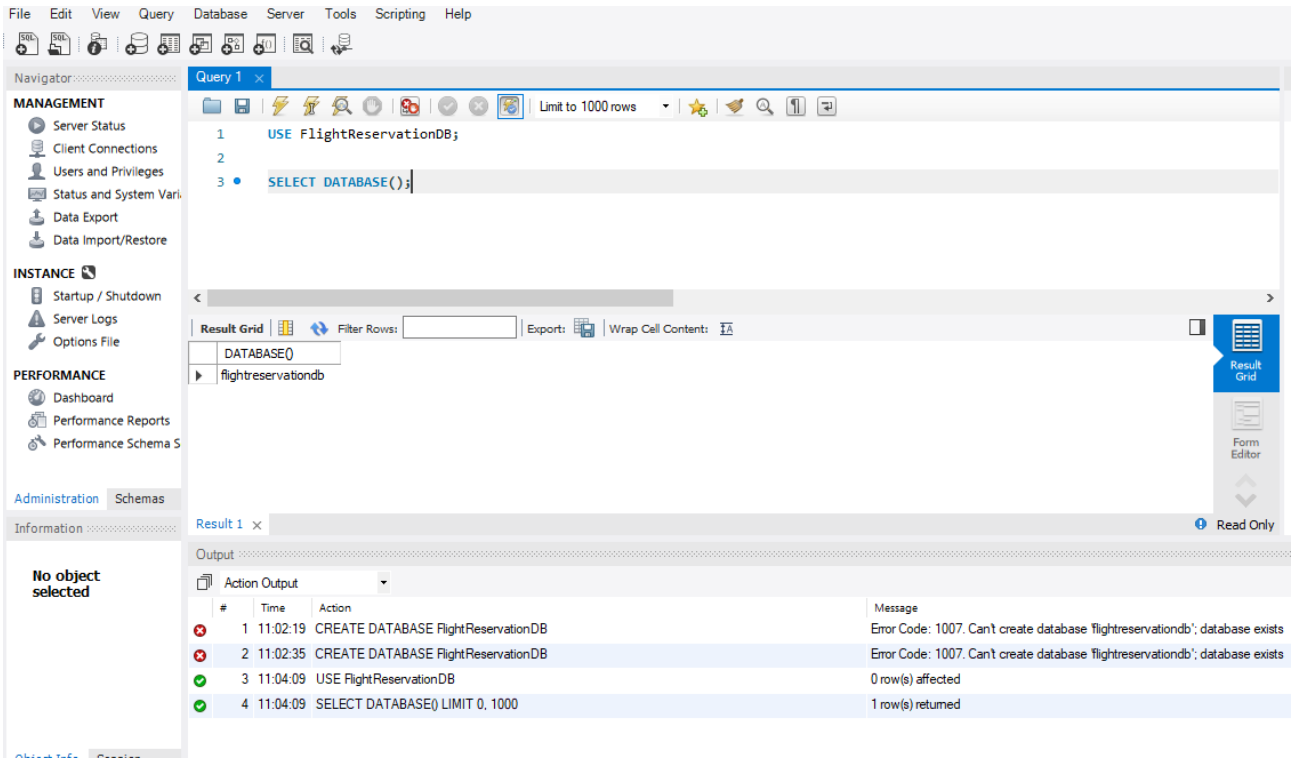
Running revenue: groups bookings by route and booking date, sums fare amounts for Confirmed/Upgraded records, then uses a windowed cumulative SUM by route.

LAG(): compares daily booking counts with the previous available date for the same route. The first date in each route partition returns NULL for the previous-day value.

9. Screenshots and Execution Evidence

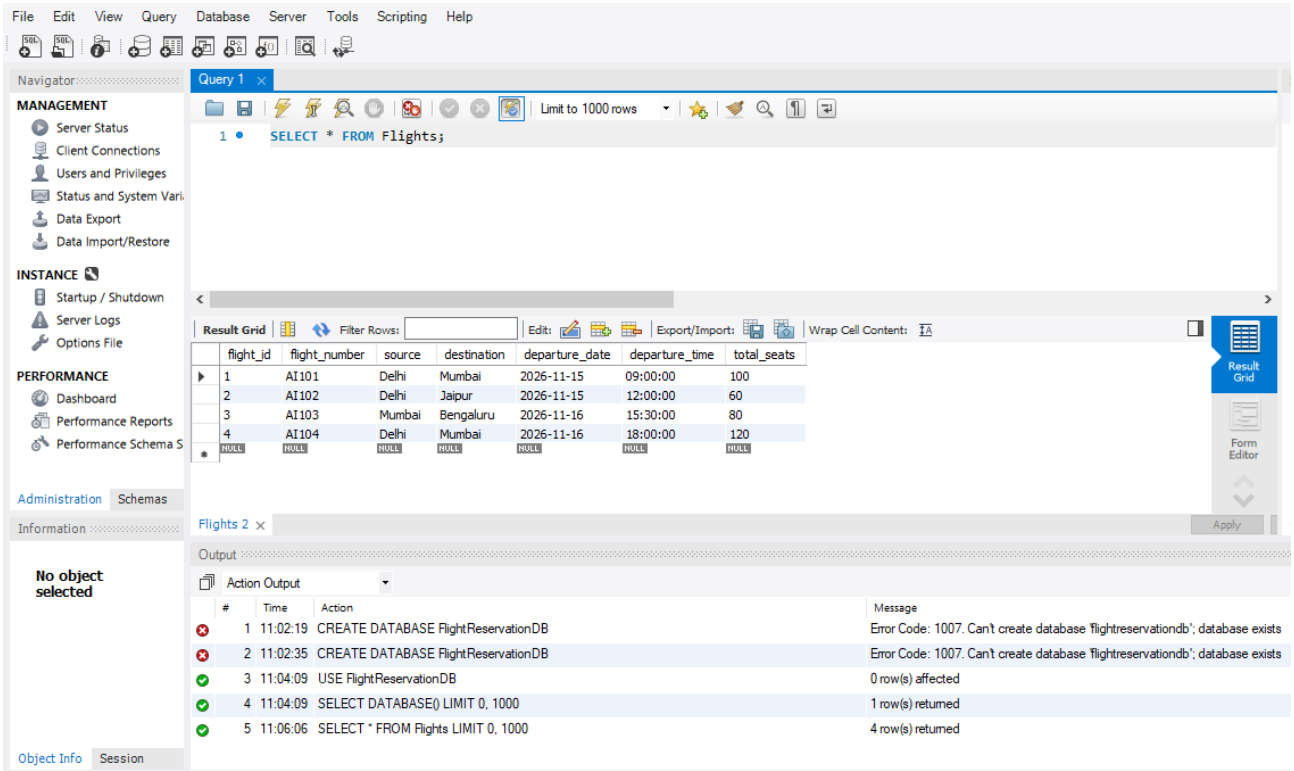
The following images are copied from the user's uploaded Word document in their original sequence. Each image is preserved as supplied; some SQL text may be small at page size, so the screenshots are supporting evidence rather than a substitute for the query descriptions above.

Screenshot 1



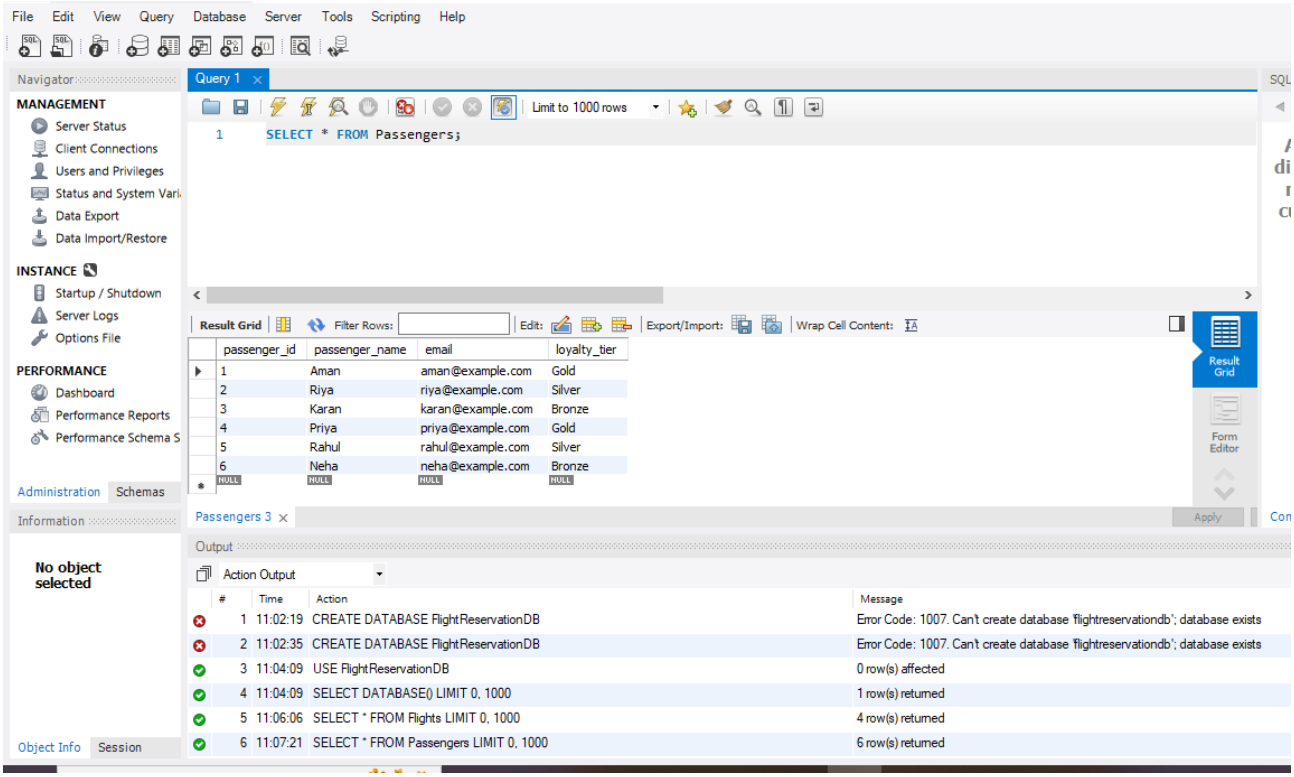
Evidence image 1 — supplied from the uploaded Workbench document.

Screenshot 2



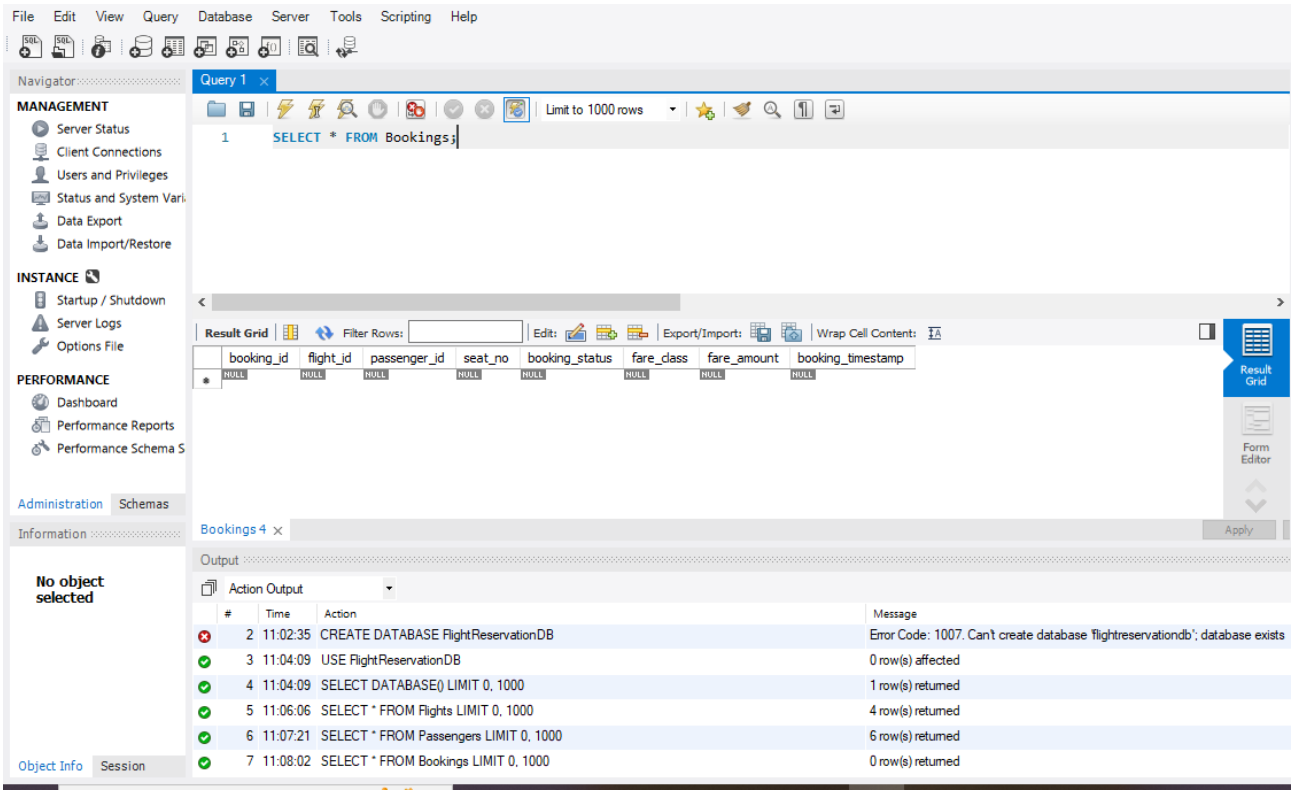
Evidence image 2 — supplied from the uploaded Workbench document.

Screenshot 3



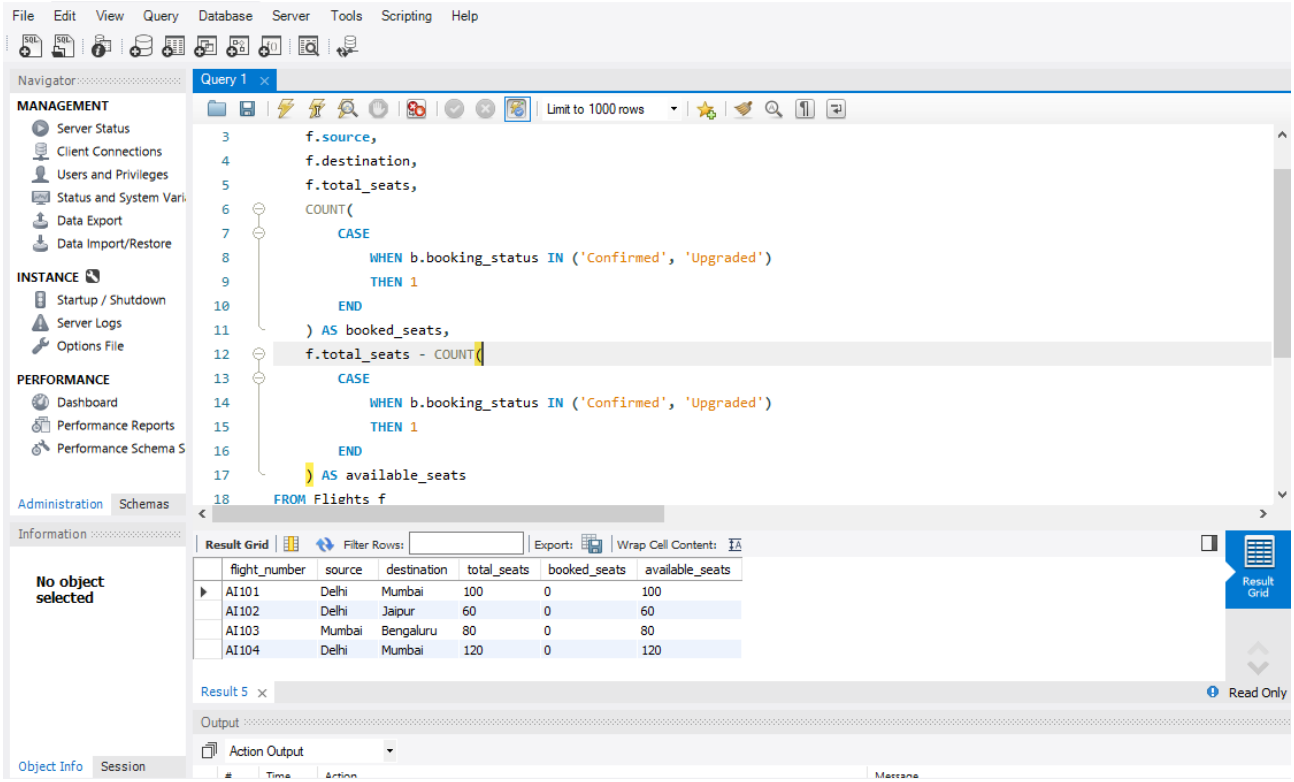
Evidence image 3 — supplied from the uploaded Workbench document.

Screenshot 4



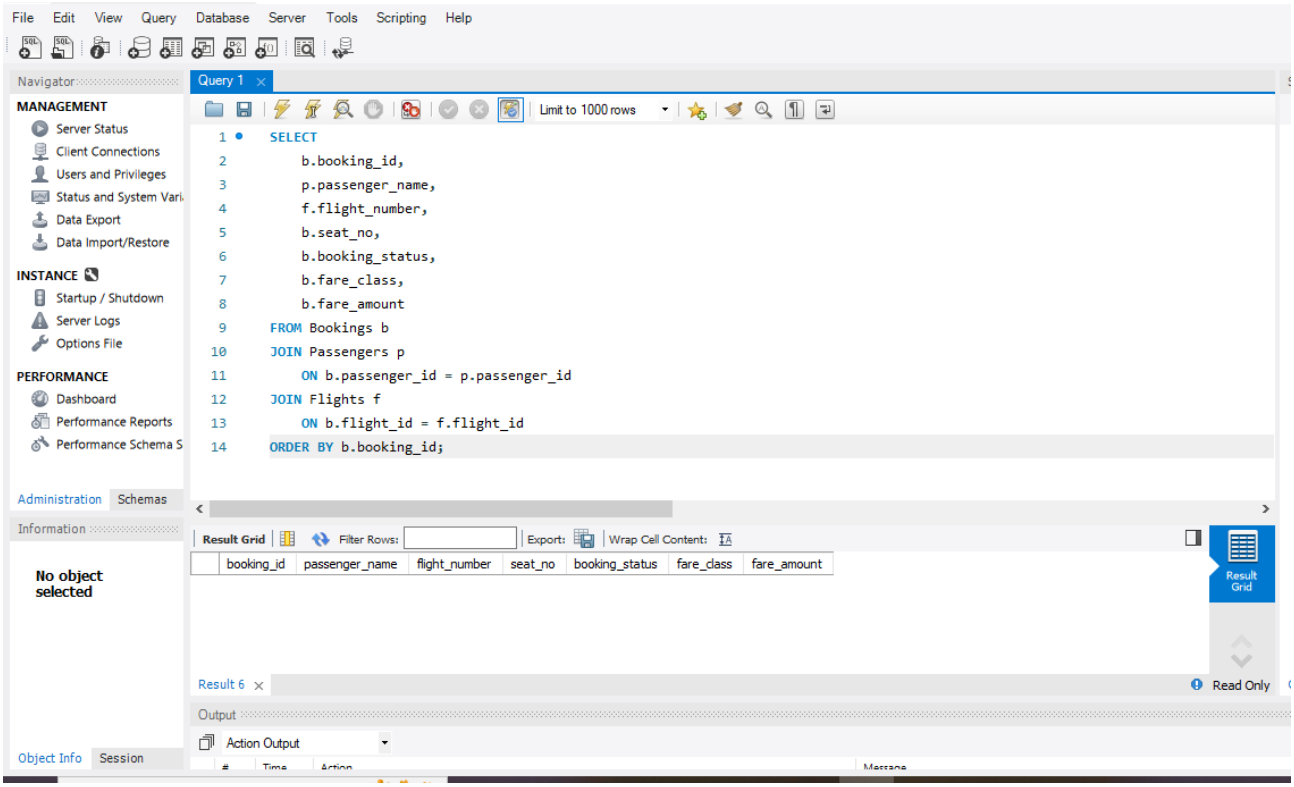
Evidence image 4 — supplied from the uploaded Workbench document.

Screenshot 5



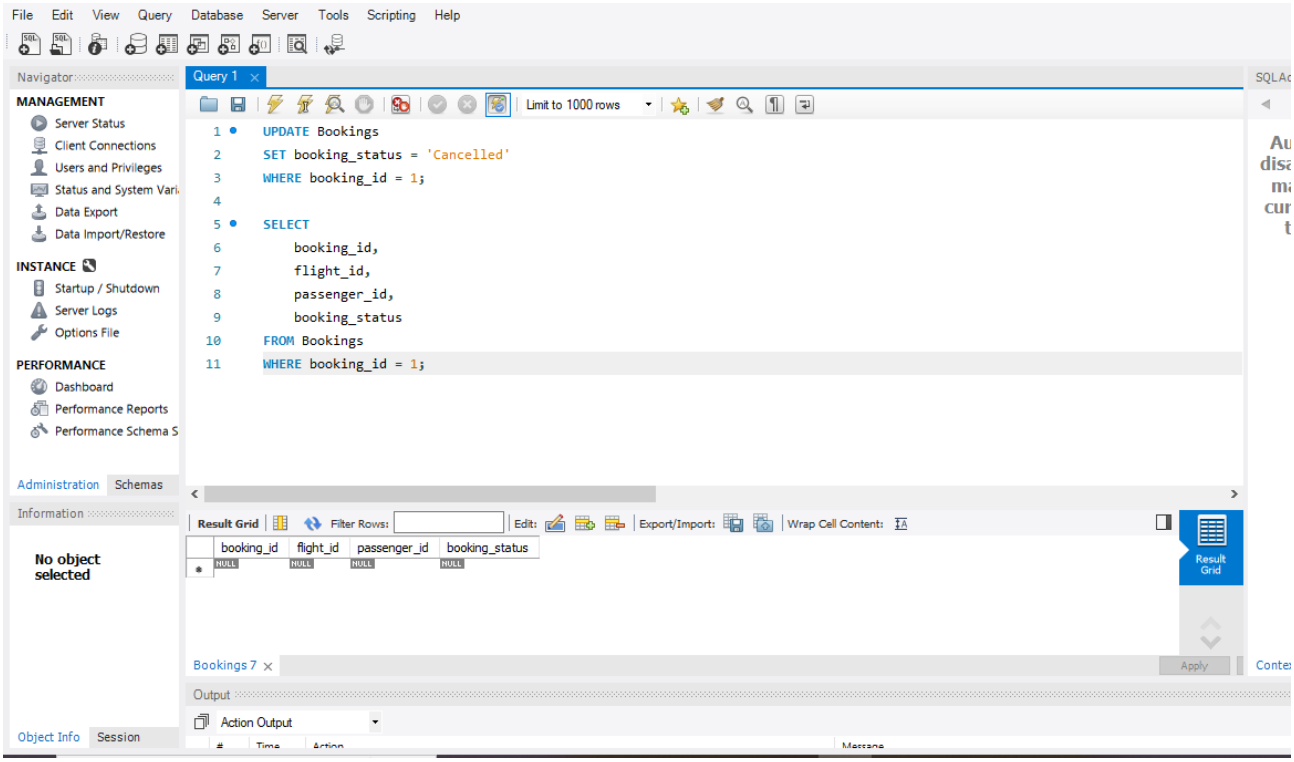
Evidence image 5 — supplied from the uploaded Workbench document.

Screenshot 6



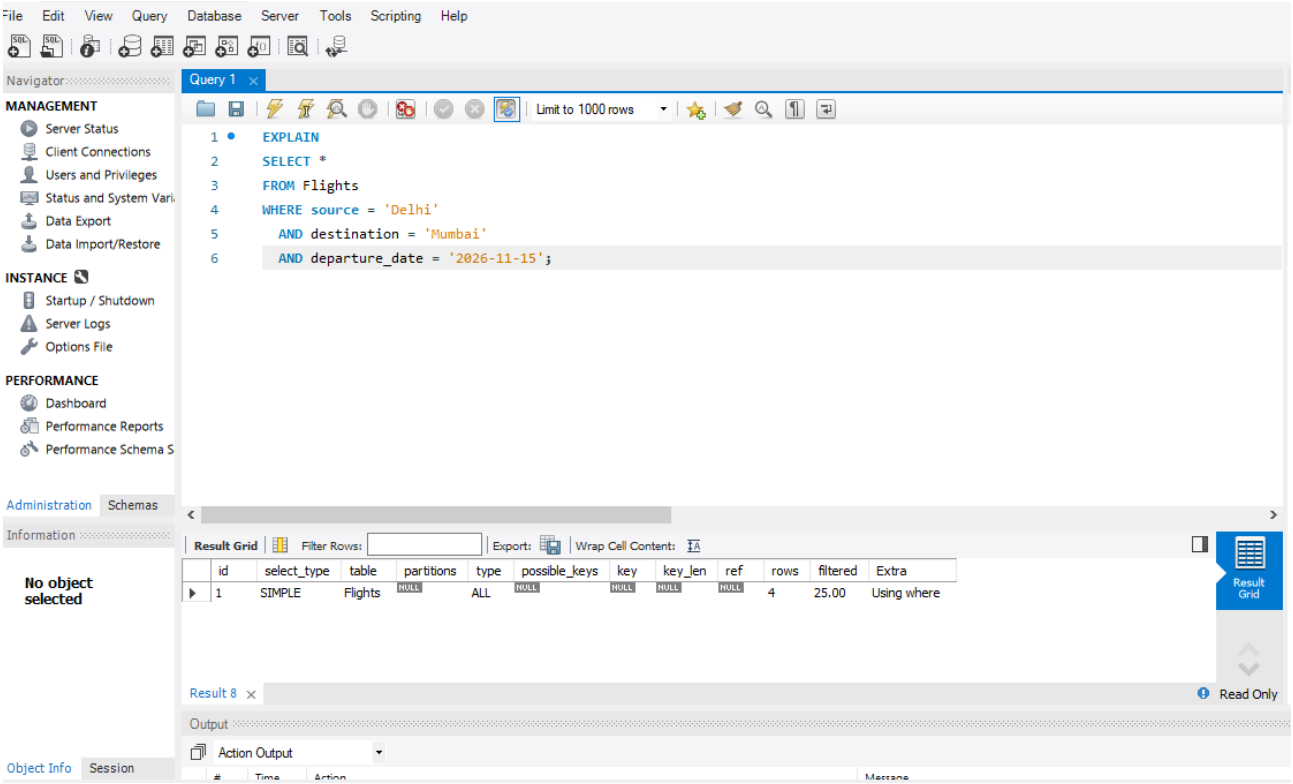
Evidence image 6 — supplied from the uploaded Workbench document.

Screenshot 7



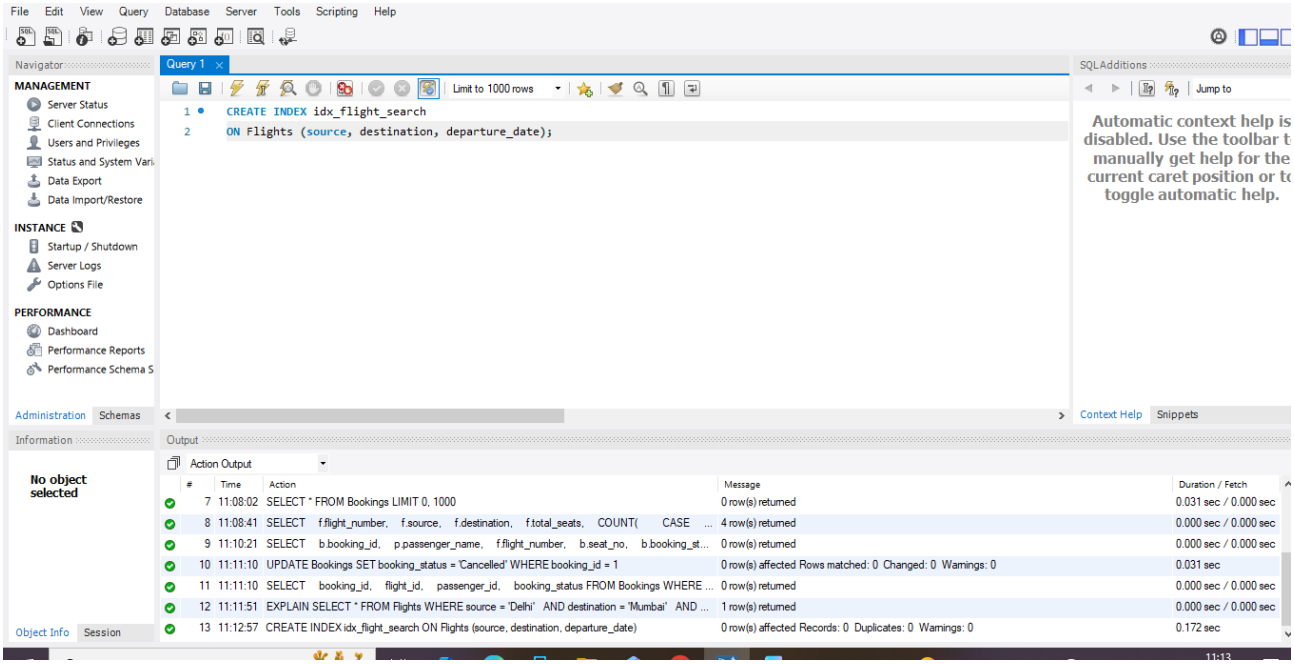
Evidence image 7 — supplied from the uploaded Workbench document.

Screenshot 8



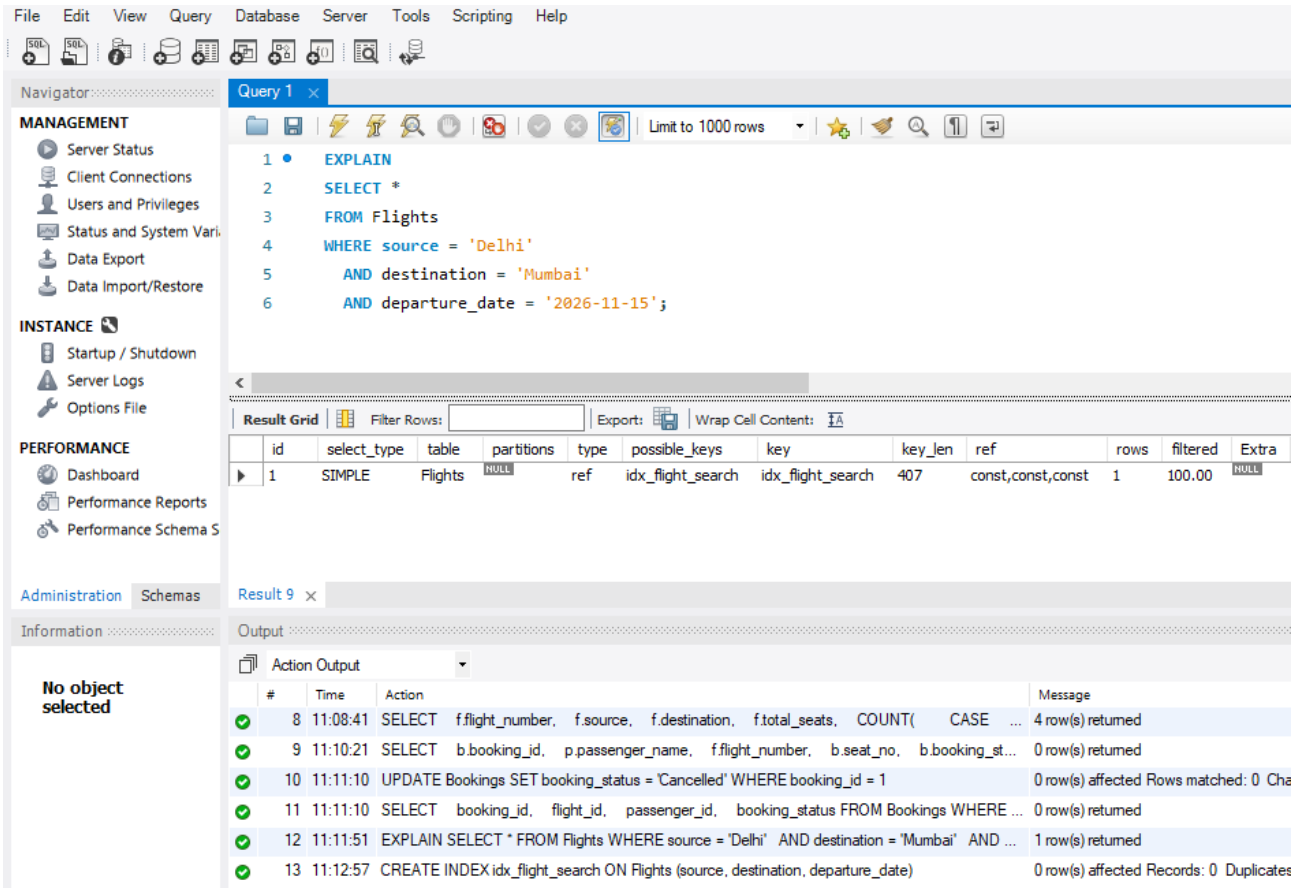
Evidence image 8 — supplied from the uploaded Workbench document.

Screenshot 9



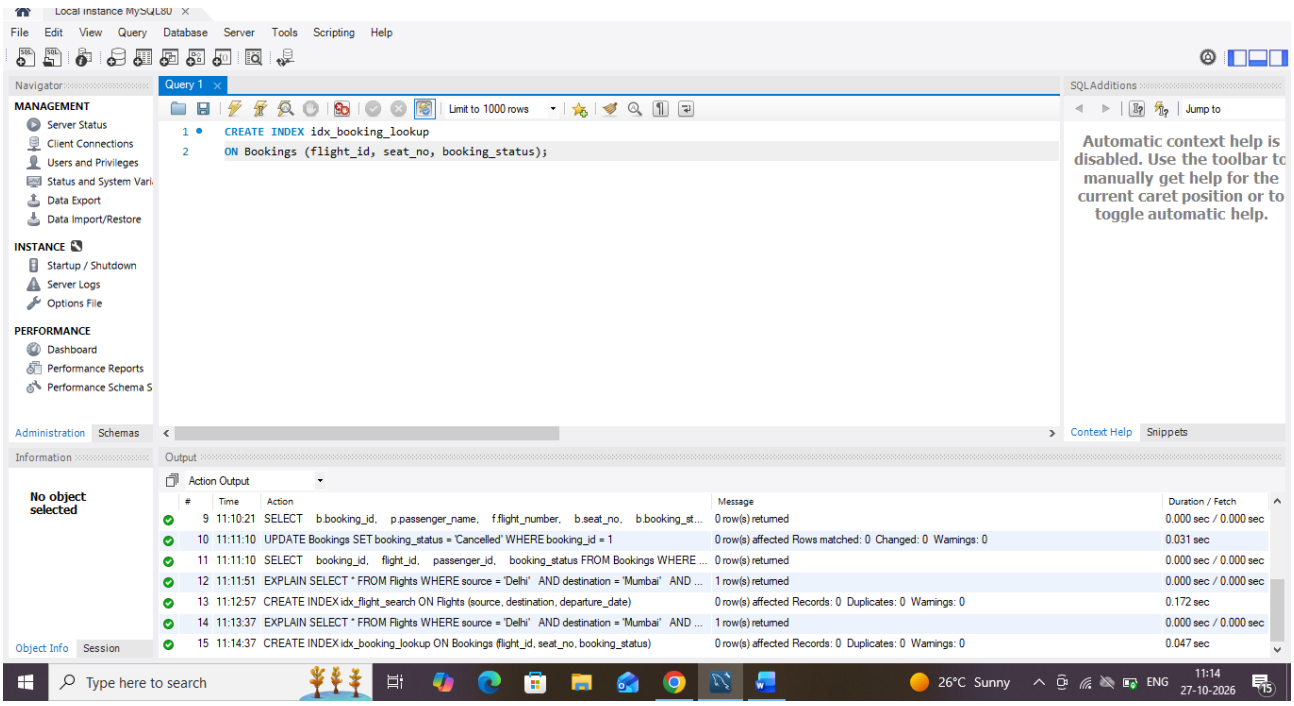
Evidence image 9 — supplied from the uploaded Workbench document.

Screenshot 10



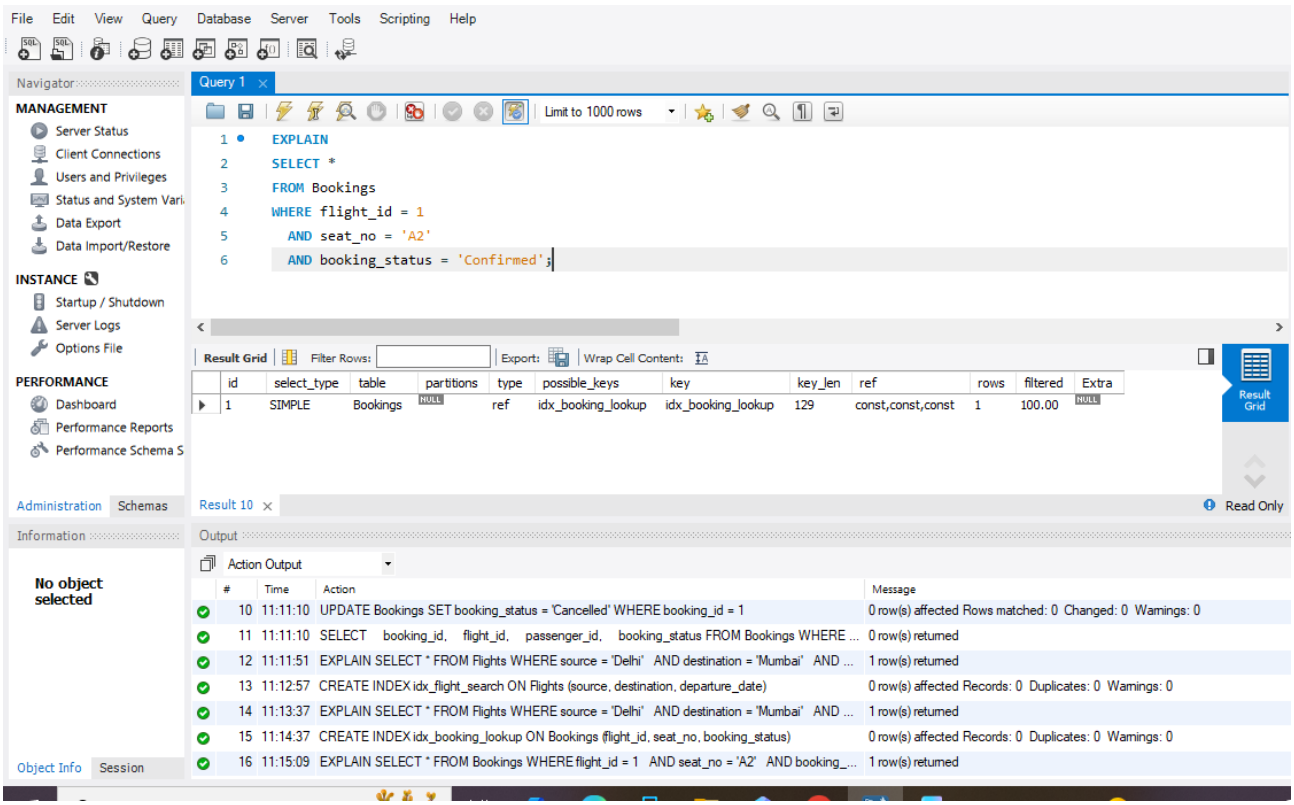
Evidence image 10 — supplied from the uploaded Workbench document.

Screenshot 11



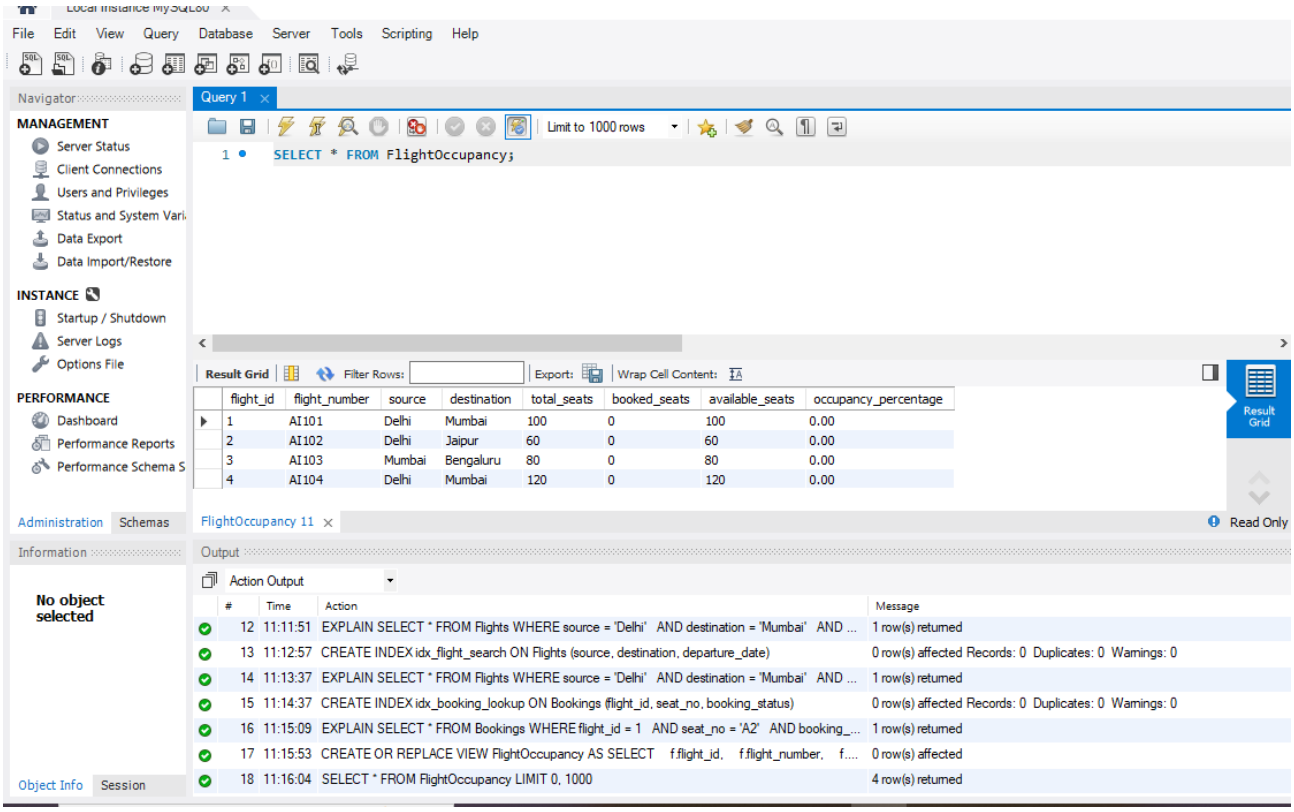
Evidence image 11 — supplied from the uploaded Workbench document.

Screenshot 12



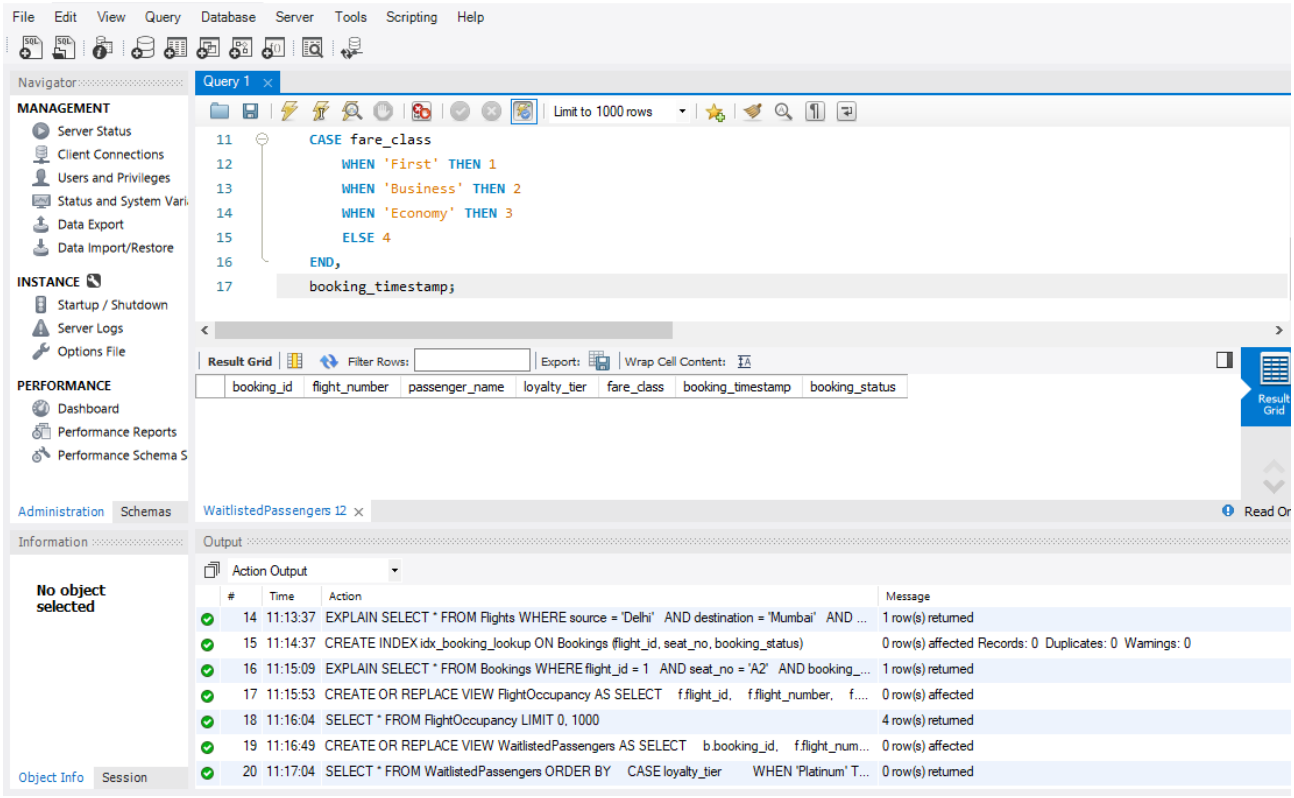
Evidence image 12 — supplied from the uploaded Workbench document.

Screenshot 13



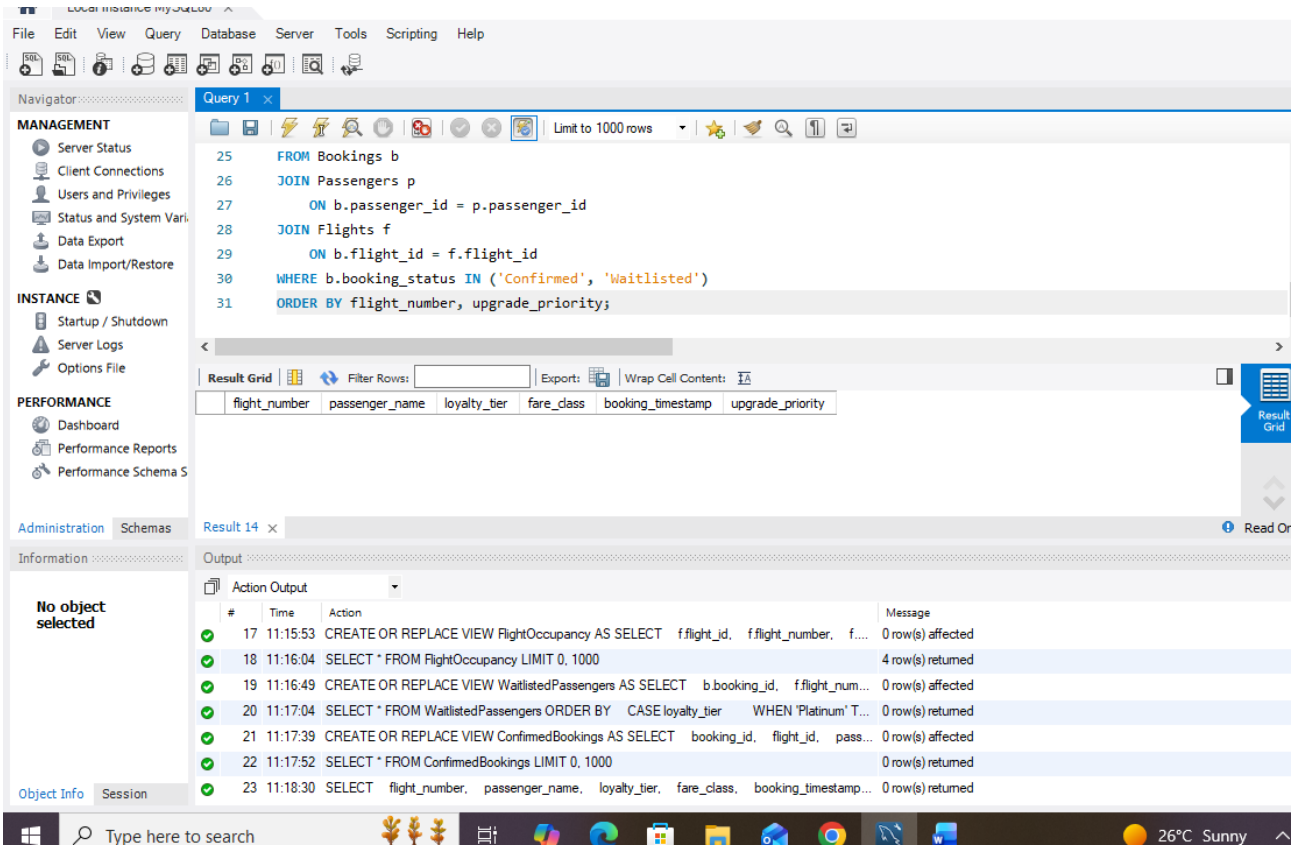
Evidence image 13 — supplied from the uploaded Workbench document.

Screenshot 14



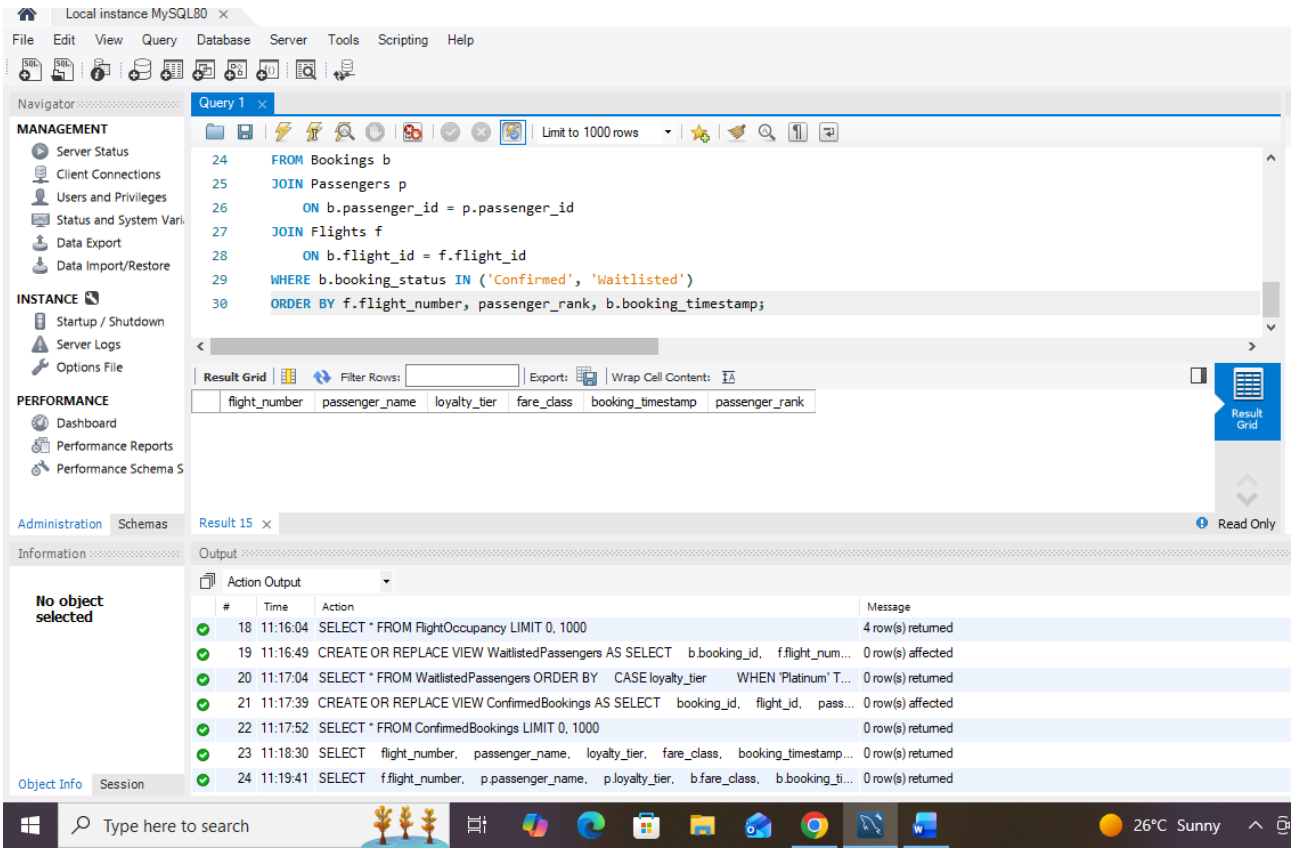
Evidence image 14 — supplied from the uploaded Workbench document.

Screenshot 15



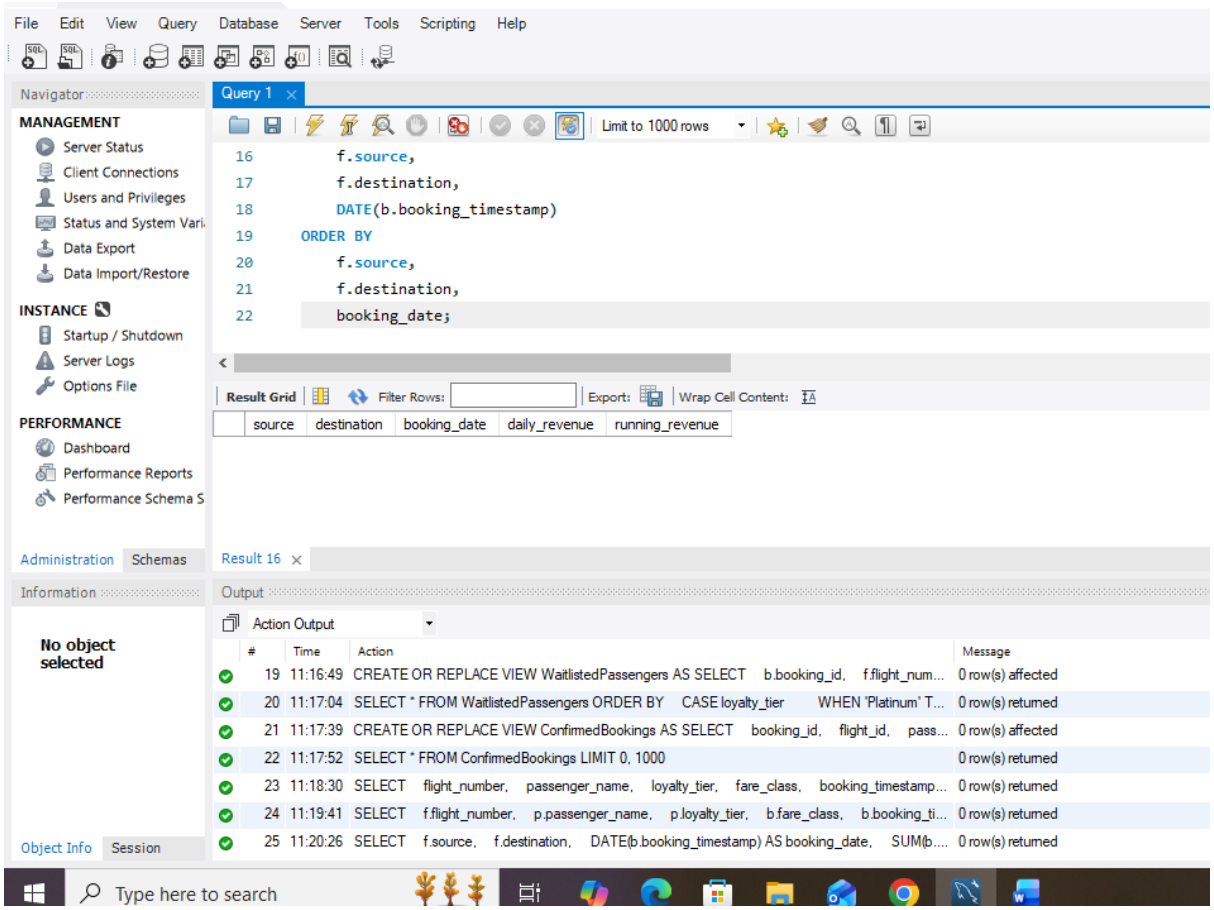
Evidence image 15 — supplied from the uploaded Workbench document.

Screenshot 16



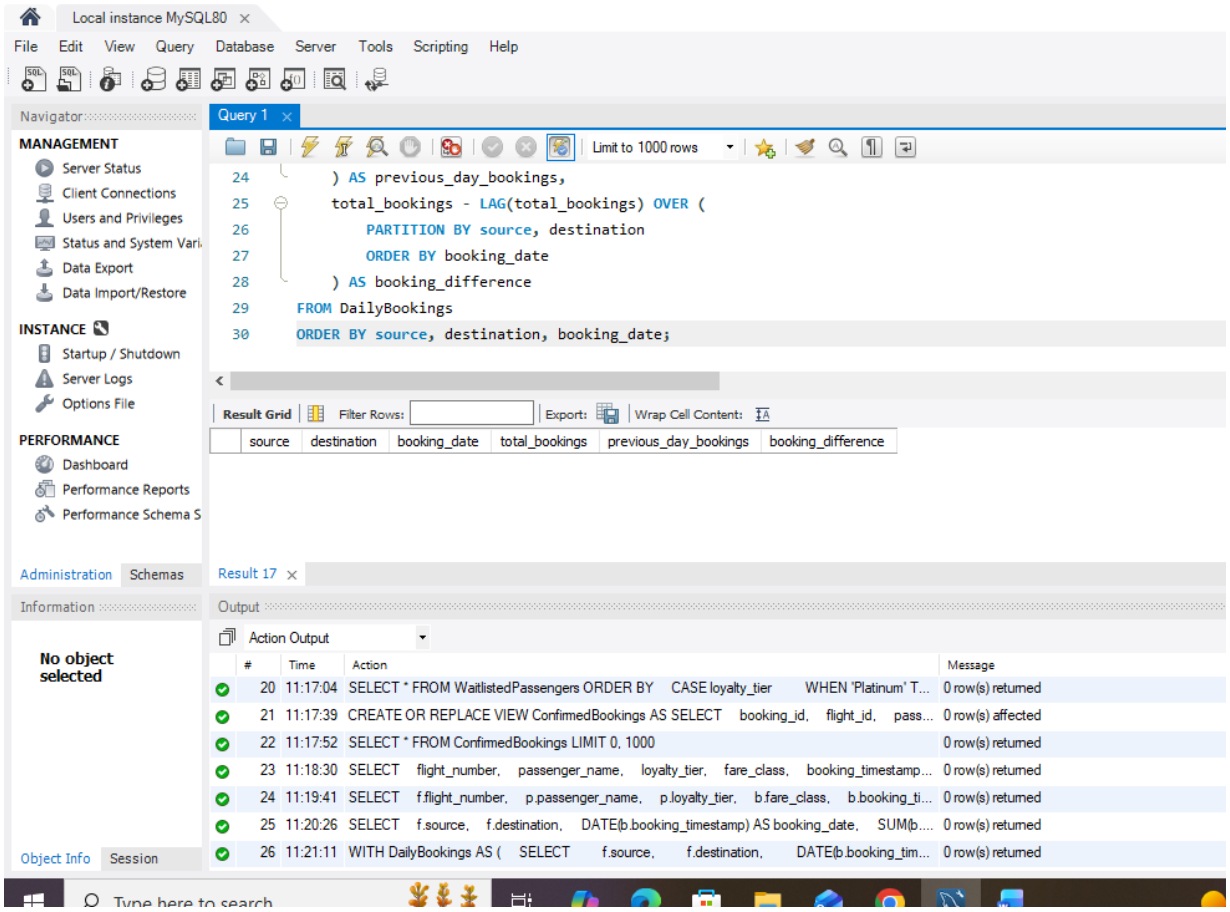
Evidence image 16 — supplied from the uploaded Workbench document.

Screenshot 17



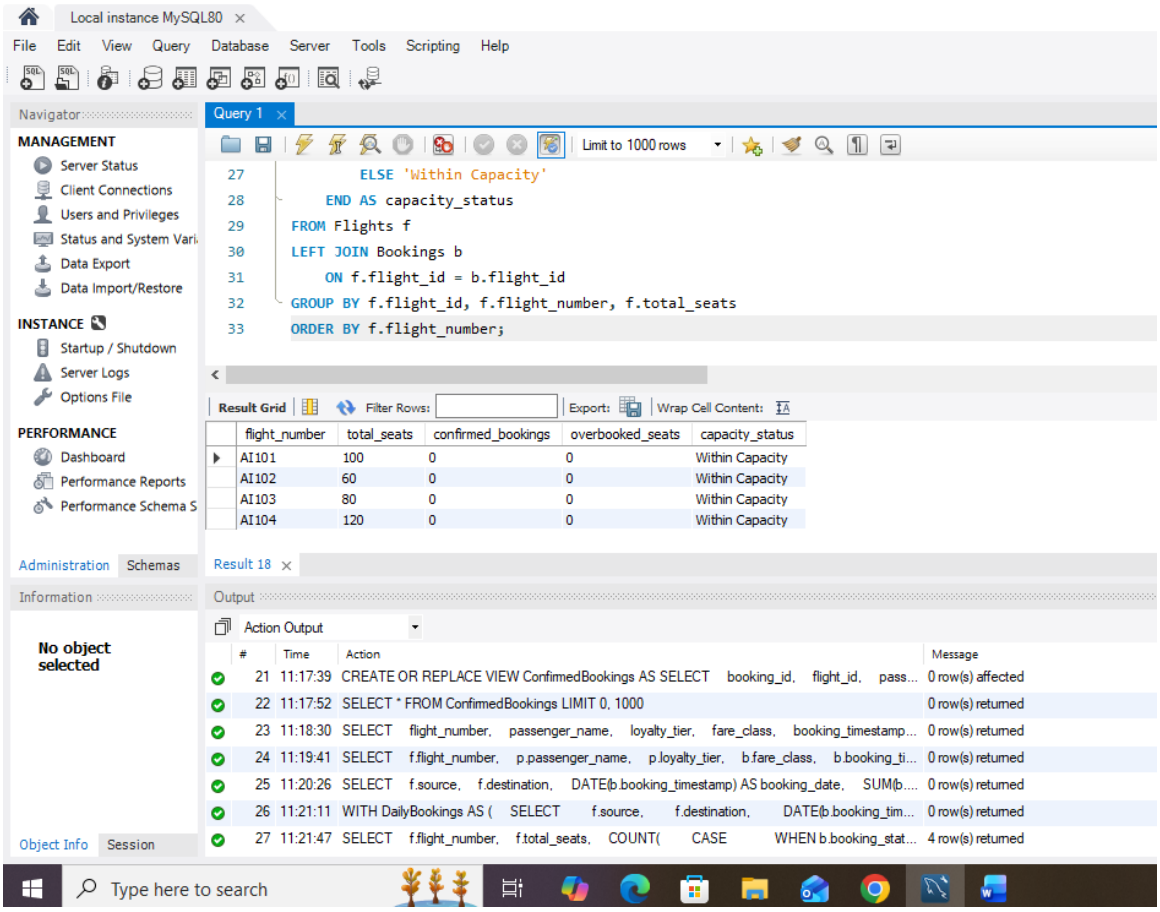
Evidence image 17 — supplied from the uploaded Workbench document.

Screenshot 18



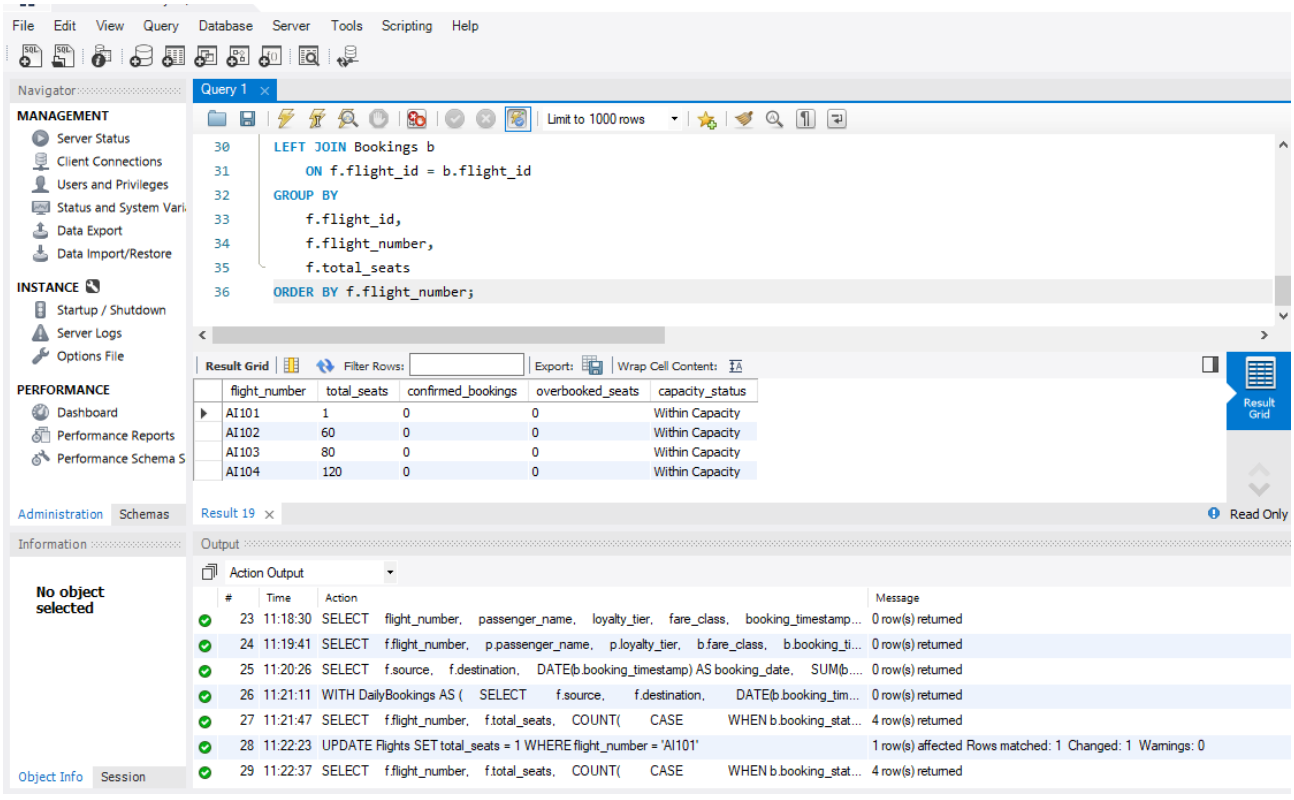
Evidence image 18 — supplied from the uploaded Workbench document.

Screenshot 19



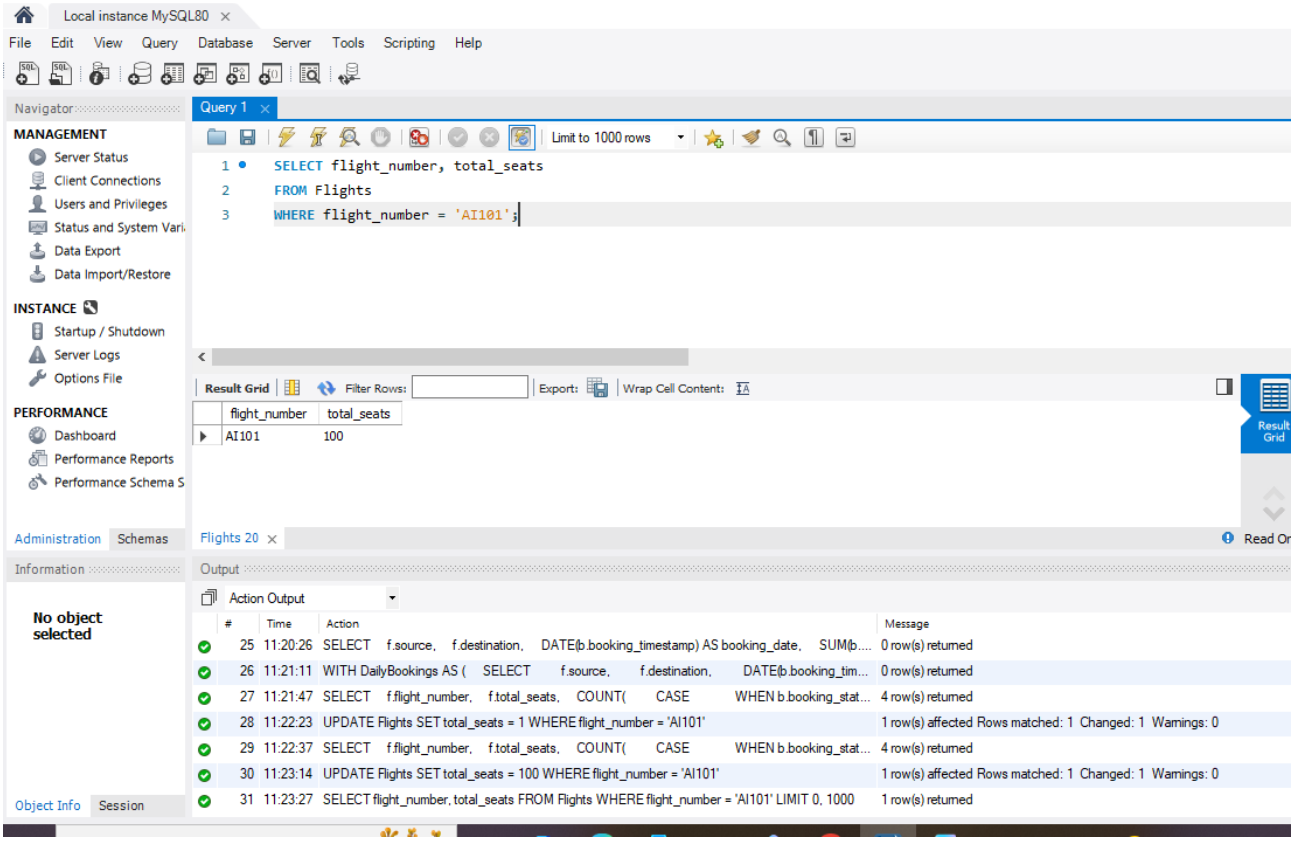
Evidence image 19 — supplied from the uploaded Workbench document.

Screenshot 20



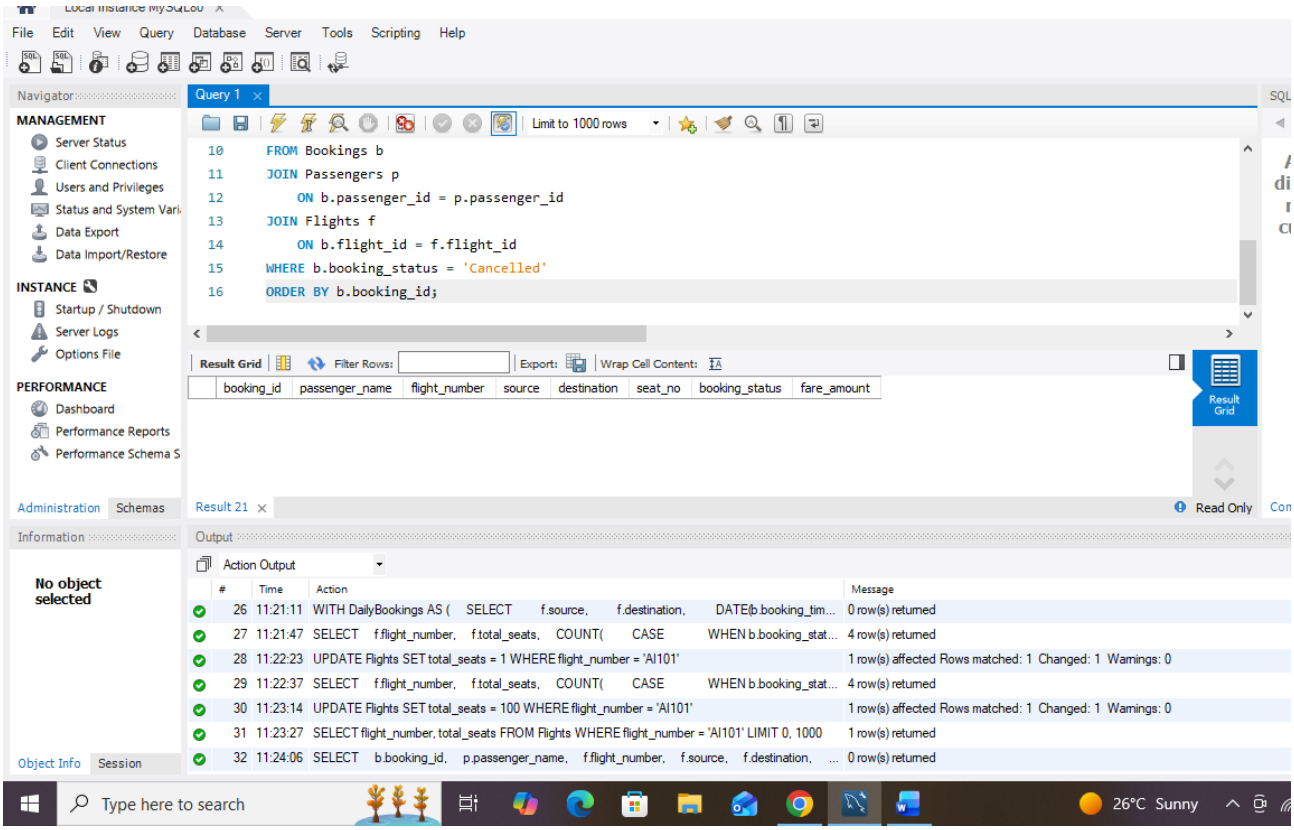
Evidence image 20 — supplied from the uploaded Workbench document.

Screenshot 21



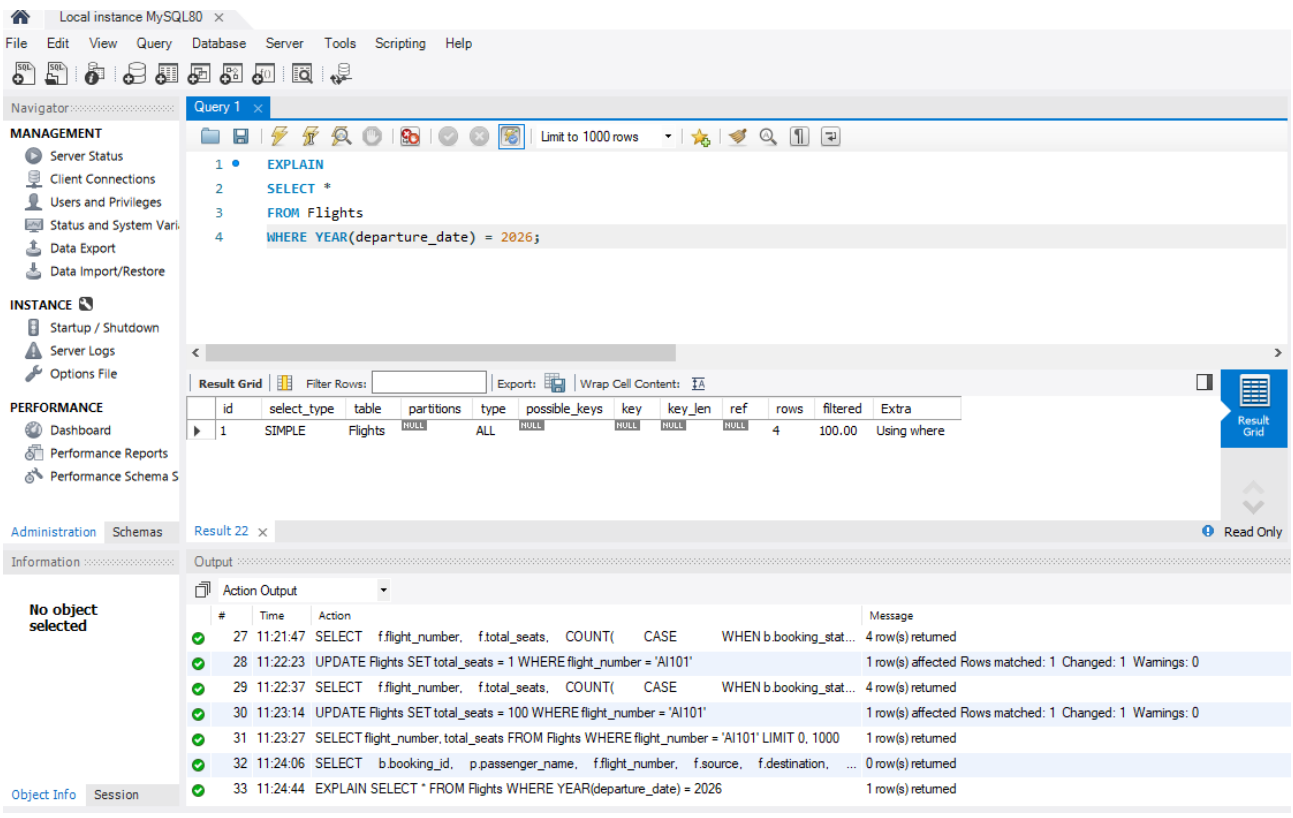
Evidence image 21 — supplied from the uploaded Workbench document.

Screenshot 22



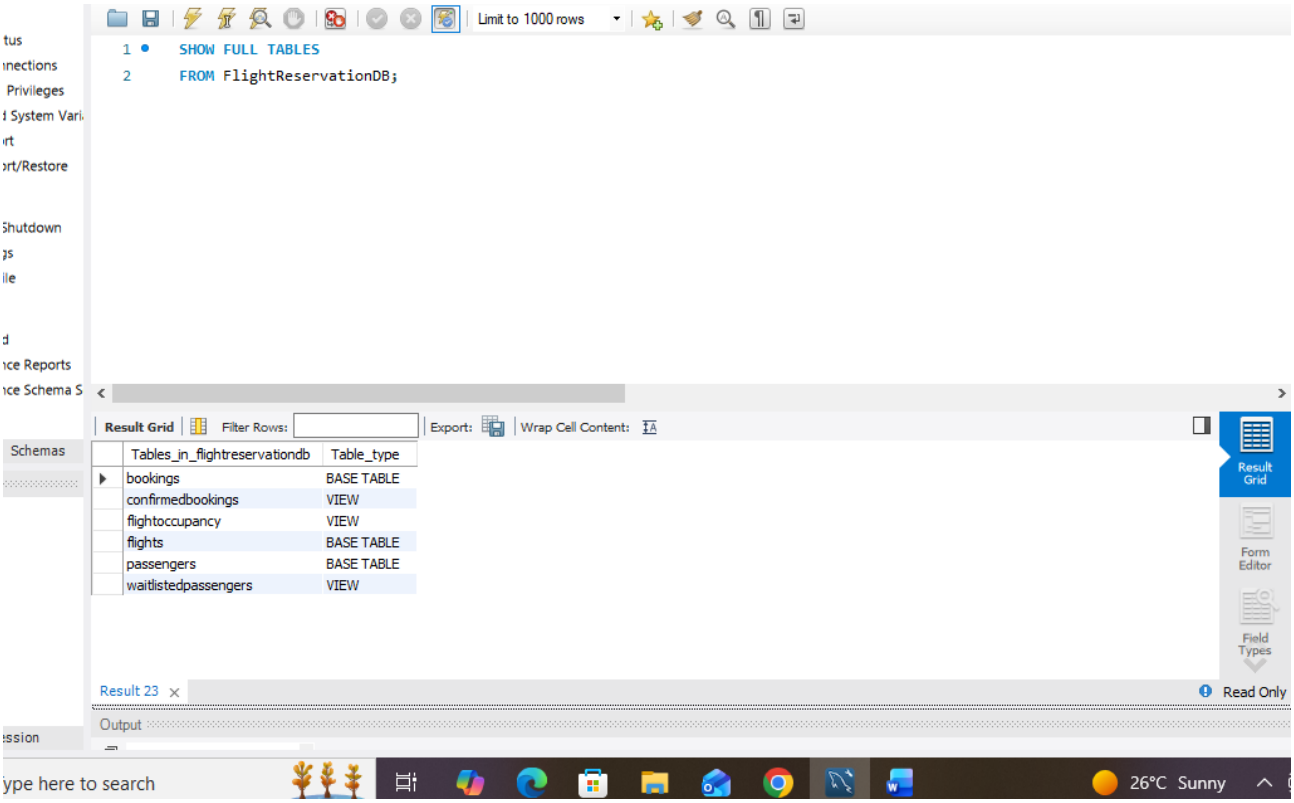
Evidence image 22 — supplied from the uploaded Workbench document.

Screenshot 23



Evidence image 23 — supplied from the uploaded Workbench document.

Screenshot 24



Evidence image 24 — supplied from the uploaded Workbench document.

10. Conclusion

This project demonstrates core DBMS concepts through a flight reservation use case: relational schema design, foreign keys, joins, booking-status reports, capacity checks, views, composite indexing, EXPLAIN, and analytical window functions. Before submission, verify that every query runs successfully in the final database and that the screenshots correspond to the final query outputs.